

Simple Lisp Programs

CS-3180

Lists

- We know how to manipulate lists
- Same rules apply to expressions
- Let's expand our knowledge
- To hopefully make more useful programs
- Less slides = more code

Variables?

- Simple != Pragmatic
- Few ways variables can be done
- We'll take some shortcuts for now
- Use the `defparameter` function
- For now, let's see how it is used

Defparameter

- Defines a global variables
- "Earmuff" notation?
- Can fill this with Symbolic Expressions
- Only use this with globals

```
1 (defparameter *variable-name* value)
2
3 (defparameter *age* 30)
4
5 (defparameter *months* (+ 3 5))
6
7 (format t "I am ~S years and ~S months old~%" *age* *months*)
```

Word of Caution

- Haven't we just written procedural code?
- Antithetical to Lisp
- Named intentionally -> "Define Parameter"
- Think of it as a parameter instead of a side-effect
- "Pollutes the global namespace"

```
1 (defparameter *age* 30)
2 (format t "I age ~S years old~%" *age*)
3
4 (setf *age* (* 5 7))
5 (format t "I am now ~S years old~%" *age*)
```

Updating "Variables"

- Setf to update a variable
- Hesitate to use this on labs
- Mutating global state
- Necessary evil for "bootstrapping" your Lisp knowledge

```
1 (defparameter *age* 30)
2 (format t "I age ~S years old~%" *age*)
3
4 (setf *age* (* 5 6))
5 (format t "I am now ~S years old~%")
```

User Input

- Lisp gives you a "simple" function
- `read-line` to read input as a string
- Code like this often leads to race conditions
- Symptomatic of a bad paradigm
- `finish-output` to block until all buffers are flushed

```
1 (format t "What is your name?~%")  
2 (finish-output)  
3 (defparameter *name* (read-line))
```

Input as Data

- You can also **read** input as data
- Executor assumes its just Lisp code
- Similar a **quote** in some ways
- Will also coerce types for you

```
1 (format t "Enter something Lisp-like: ")
2 (finish-output)
3 (defparameter *code* (read))
4
5 (format t "You entered: ~S~%" *code*)
6 (format t "The type was: ~S~%" (type-of *code*))
```

If "Statements"

- `if` are really just functions
- "If construct actually" - 🧐
- More complex forms later
- Lazily evaluated
- Let's see this

```
(if test-form then-form [optional-else])
```

If Construct

- Selective Indentation

```
1 (format t "How old are you?~%")
2 (finish-output)
3 (defparameter *age* (read))
4
5 (if (>= *age* 35)
6     (format t "You are old enough to be president"))
```

```
1 (format t "How old are you?~%")
2 (finish-output)
3 (defparameter *age* (read))
4
5 (if (>= *age* 35)
6     (format t "You ARE old enough to be president")
7     (format t "You are NOT old enough to be president"))
8
```

Functions

- Define function via `defun`
- Parameters are just a list
- Scoped to the body -> Important
- Evaluate to their LAST expression evaluated
- Let's see how this works

```
(defun function-name (parameter-list)  
  body-expression)
```

Functions

- Doc String as a parameter

```
1 (defun square (x)
2   (* x x))
3
4 (format t "~S~%" (square 5))
5
6 ; / will just return a "ratio" if both values are ints
7 (defun do-some-math (value divisor)
8   (/ (square value) (float divisor)))
9
10 (format t "~S~%" (do-some-math 5 2))
11
12 (defun cube (value)
13   "Returns the cube of the number"
14   (* value value value))
15
```

Summary

- `defparameter` and `setf` for globals
- Pollute the namespace
- `if` function for simple conditionals
- `defun` for functions